

DPDK Mempool实现

[Mempool的设计](#)

[Packet MBuf介绍](#)

[Mempool的handler实现](#)

[Mempool的cache实现](#)

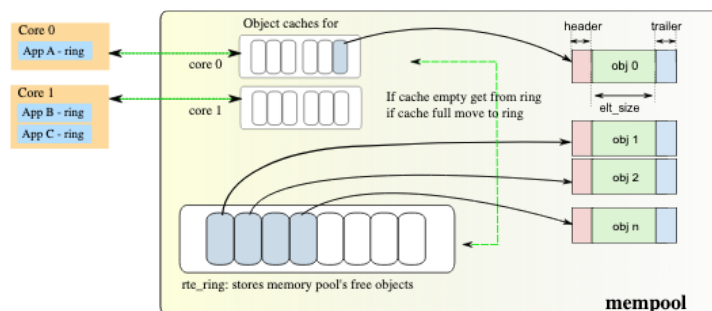
[参考链接](#)

Mempool的设计

内存池（Memory Pool） 是一种用于分配固定大小对象的分配器。在 DPDK 中，内存池通过名称进行标识，并使用一个 **mempool handler（内存池处理器）** 来存储空闲对象。默认的内存池处理器是基于环形队列（ring-based）的。

它还提供一些可选功能，例如：

- **每个核心的对象缓存（per-core object cache）**，
- **对齐辅助功能（alignment helper）**，用于确保对象被适当填充，从而在所有 DRAM 或 DDR3 通道上均匀分布。

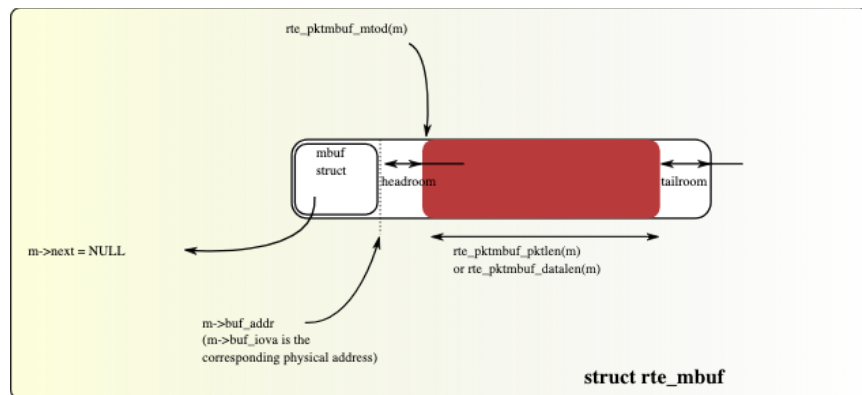


A mempool in Memory with its Associated Ring

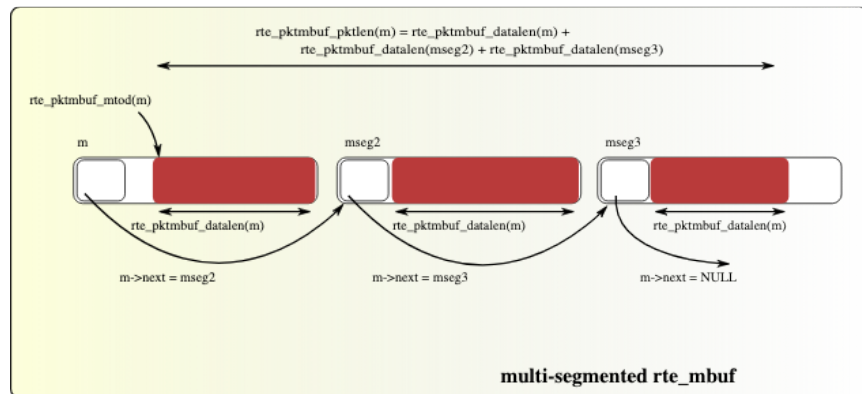
Packet MBuf介绍

Packet（MBuf）库提供了分配和释放缓冲区（mbuf）的能力，DPDK 应用程序可以使用这些缓冲区来存储消息数据。消息缓冲区通过**内存池库（Memory Pool Library）**存储在一个内存池（mempool）中。

一个 `rte_mbuf` 结构体通常用于承载网络数据包缓冲区，但实际上它也可以用于存储任何类型的数据（例如控制数据、事件等）。`rte_mbuf` 的头部结构被设计得尽可能小，目前仅占用两个缓存行（cache line），其中最常用的字段被放在第一个缓存行中，以提高访问效率



An mbuf with One Segment



An mbuf with Three Segments

Mempool的handler实现

DPDK Mempool 可以使用外部内存子系统，例如外部硬件内存管理系统或基于软件的内存分配器。

一个 mempool handler（内存池处理器）有两个关键方面：

1. 添加你自定义的 mempool 操作（ops）代码。

这可以通过添加一个新的 mempool ops 代码，并使用 `RTE_MEMPOOL_REGISTER_OPS` 宏来实现。

2. 使用新的 API 创建和配置内存池。

通过调用 `rte_mempool_create_empty()` 创建一个新的空内存池，然后使用 `rte_mempool_set_ops_byname()` 指定要使用的 ops（操作集）。

在同一个应用程序中，可以使用多个不同的 mempool handler。你可以先通过

`rte_mempool_create_empty()` 创建一个新的内存池，然后使用

`rte_mempool_set_ops_byname()` 将该内存池与相应的 mempool handler 回调结构（ops）关联起来。

对于旧版应用程序，仍然可以继续使用旧的 `rte_mempool_create()` 接口，该接口默认使用基于 ring 的内存池处理器。如果这些应用希望使用新的 mempool handler，则需要对其进行相应修改。

```
1 /* mbuf defines */
2 #define RTE_MBUF_DEFAULT_MEMPOOL_OPS "ring_mp_mc"
```

DPDK内置了四种mempool handler的实现模式，默认为ring_mp_mc多生产多消费模式：

- ring_sp_sc单生产单消费模式
- ring_sp_mc单生产多消费模式
- ring_mp_sc多生产单消费模式
- ring_mp_mc多生产多消费模式（默认）

```
1 /* create the mempool */
2 struct rte_mempool *
3 rte_mempool_create(const char *name, unsigned n, unsigned elt_size,
4 unsigned cache_size, unsigned private_data_size,
5 rte_mempool_ctor_t *mp_init, void *mp_init_arg,
6 rte_mempool_obj_cb_t *obj_init, void *obj_init_arg,
7 int socket_id, unsigned flags)
8 {
9     ...
10    mp = rte_mempool_create_empty(name, n, elt_size, cache_size,
11    private_data_size, socket_id, flags);
12    if (mp == NULL)
13        return NULL;
14
15    /*
16     * Since we have 4 combinations of the SP/SC/MP/MC examine the flags to
17     * set the correct index into the table of ops structs.
18     */
19    if ((flags & MEMPOOL_F_SP_PUT) && (flags & MEMPOOL_F_SC_GET))
20        ret = rte_mempool_set_ops_byname(mp, "ring_sp_sc", NULL);
21    else if (flags & MEMPOOL_F_SP_PUT)
22        ret = rte_mempool_set_ops_byname(mp, "ring_sp_mc", NULL);
23    else if (flags & MEMPOOL_F_SC_GET)
24        ret = rte_mempool_set_ops_byname(mp, "ring_mp_sc", NULL);
25    else
26        ret = rte_mempool_set_ops_byname(mp, "ring_mp_mc", NULL);
27
28    ...
29 }
```

Mempool的cache实现

从 CPU 使用率的角度来看，多个核心同时访问一个内存池中用于存放空闲缓冲区的 ring 时开销可能很高，因为每次访问都需要执行一次 CAS（Compare-And-Set）操作。

为了避免过多的访问请求直接打到内存池的 ring 上，内存池分配器可以为每个核心维护一个 **per-core cache（每核缓存）**，并通过这个缓存以批量方式访问 ring，从而大大减少对内存池结构的

加锁操作。

这样，每个核心就可以独立访问其自身的缓存（无锁），只有当缓存用尽时，该核心才需要从内存池的 ring 中获取更多对象；当缓存太满时，则将多余的对象归还给 ring。

尽管这种机制可能导致某些核心的缓存中存在一部分“闲置”的缓冲区，但由于核心可以无需加锁地快速访问自身缓存，从而带来了显著的性能提升。

这个缓存由一个 **每核的小型指针表** 和一个作为栈使用的 **长度字段** 组成。在创建内存池时，可以选择是否启用这个内部缓存。

缓存的最大容量是静态定义的，编译时通过宏 `RTE_MEMPOOL_CACHE_MAX_SIZE` 来指定。

一下向mempool中消费objs的代码实现，首先检查local cache中是否存在足够的objs，如果可以直接使用cache中的，则不用从mempool的ring中申请：

```
1 static __rte_always_inline int
2 rte_mempool_get_bulk(struct rte_mempool *mp, void **obj_table, unsigned int n)
3 {
4     struct rte_mempool_cache *cache;
5     cache = rte_mempool_default_cache(mp, rte_lcore_id());
6     rte_mempool_trace_get_bulk(mp, obj_table, n, cache);
7     return rte_mempool_generic_get(mp, obj_table, n, cache);
8 }
```

```
1 static __rte_always_inline int
2 __mempool_generic_get(struct rte_mempool *mp, void **obj_table,
3     unsigned int n, struct rte_mempool_cache *cache)
4 {
5     int ret;
6     uint32_t index, len;
7     void **cache_objs;
8
9     /* No cache provided or cannot be satisfied from cache */
10    if (unlikely(cache == NULL || n >= cache->size))
11        goto ring_dequeue;
12
13    cache_objs = cache->objs;
14
15    /* Can this be satisfied from the cache? */
16    if (cache->len < n) {
17        /* No. Backfill the cache first, and then fill from it */
18        uint32_t req = n + (cache->size - cache->len);
19
20        /* How many do we require i.e. number to fill the cache + the request */
21        ret = rte_mempool_ops_dequeue_bulk(mp,
22            &cache->objs[cache->len], req);
23        if (unlikely(ret < 0)) {
24            /*
25             * In the off chance that we are buffer constrained,
26             * where we are not able to allocate cache + n, go to
27             * the ring directly. If that fails, we are truly out of
```

```

28     * buffers.
29     */
30     goto ring_dequeue;
31 }
32
33     cache->len += req;
34 }
35
36     /* Now fill in the response ... */
37     for (index = 0, len = cache->len - 1; index < n; ++index, len--, obj_table++)
38         *obj_table = cache_objs[len];
39
40     cache->len -= n;
41
42     __MEMPOOL_STAT_ADD(mp, get_success, n);
43
44     return 0;
45
46 ring_dequeue:
47
48     /* get remaining objects from ring */
49     ret = rte_mempool_ops_dequeue_bulk(mp, obj_table, n);
50
51     if (ret < 0)
52         __MEMPOOL_STAT_ADD(mp, get_fail, n);
53     else
54         __MEMPOOL_STAT_ADD(mp, get_success, n);
55
56     return ret;
57 }
58

```

以下是向mempool生产objs的代码实现，如果local cache中obj没有超过刷新的阈值（1.5 x cache size），则直接缓存在cache中，超过则向mempool的ring中提交：

```

1 static __rte_always_inline void
2 rte_mempool_put_bulk(struct rte_mempool *mp, void * const *obj_table,
3                     unsigned int n)
4 {
5     struct rte_mempool_cache *cache;
6     cache = rte_mempool_default_cache(mp, rte_lcore_id());
7     rte_mempool_trace_put_bulk(mp, obj_table, n, cache);
8     rte_mempool_generic_put(mp, obj_table, n, cache);
9 }

```

```

1 static __rte_always_inline void
2 __mempool_generic_put(struct rte_mempool *mp, void * const *obj_table,
3                      unsigned int n, struct rte_mempool_cache *cache)
4 {
5     void **cache_objs;
6
7     /* increment stat now, adding in mempool always success */
8     __MEMPOOL_STAT_ADD(mp, put, n);
9
10    /* No cache provided or if put would overflow mem allocated for cache */
11    if (unlikely(cache == NULL || n > RTE_MEMPOOL_CACHE_MAX_SIZE))
12        goto ring_enqueue;

```

```

13
14  cache_objs = &cache->objs[cache->len];
15
16  /*
17   * The cache follows the following algorithm
18   * 1. Add the objects to the cache
19   * 2. Anything greater than the cache min value (if it crosses the
20   *    cache flush threshold) is flushed to the ring.
21   */
22
23  /* Add elements back into the cache */
24  rte_memcpy(&cache_objs[0], obj_table, sizeof(void *) * n);
25
26  cache->len += n;
27
28  if (cache->len >= cache->flushthresh) {
29      rte_mempool_ops_enqueue_bulk(mp, &cache->objs[cache->size],
30          cache->len - cache->size);
31      cache->len = cache->size;
32  }
33
34  return;
35
36 ring_enqueue:
37
38  /* push remaining objects in ring */
39  #ifdef RTE_LIBRTE_MEMPOOL_DEBUG
40      if (rte_mempool_ops_enqueue_bulk(mp, obj_table, n) < 0)
41          rte_panic("cannot put objects in mempool\n");
42  #else
43      rte_mempool_ops_enqueue_bulk(mp, obj_table, n);
44  #endif
45  }

```

参考链接

[🔗 源码分析: Mempool — zzq's blog](#)

[🔗 2. Memory Pool Library — Data Plane Development Kit 25.07.0 documentation](#)

[🔗 2. Memory Pool Library — Data Plane Development Kit 25.07.0 documentation](#)